AI Knowledge & Support Assistant

A RAG assistant that answers from a help centre, cites its sources, and refuses when it does not know, measured on held-out questions.

Kriti Singhal · Technical documentation · October 2026

GitHub: github.com/krit566/ai-support-assistant

Contents

1. Overview

2. How a question is answered

3. Evaluation

4. API and operations

5. Code map

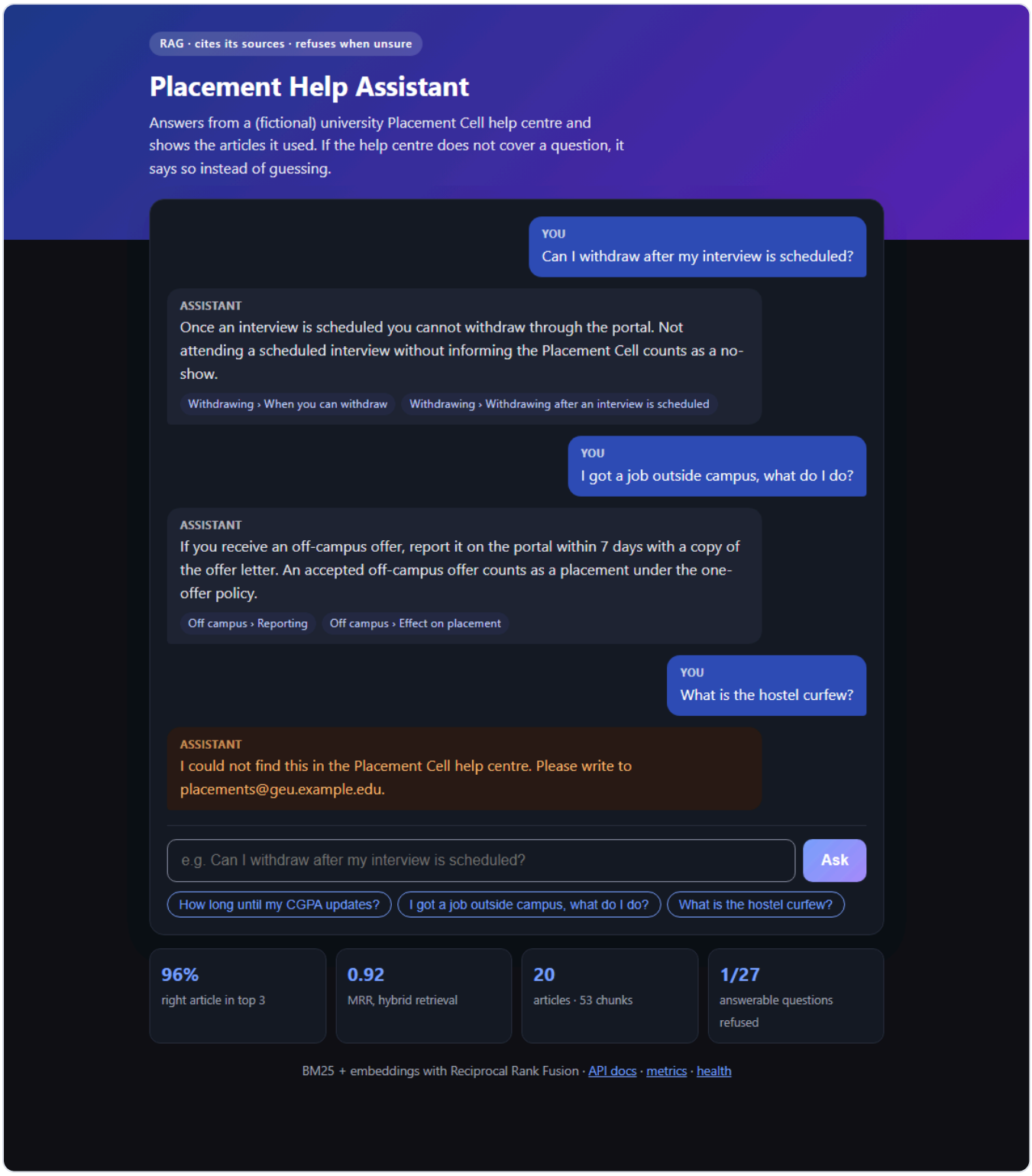
6. Next steps

7. Interview questions

1 Overview

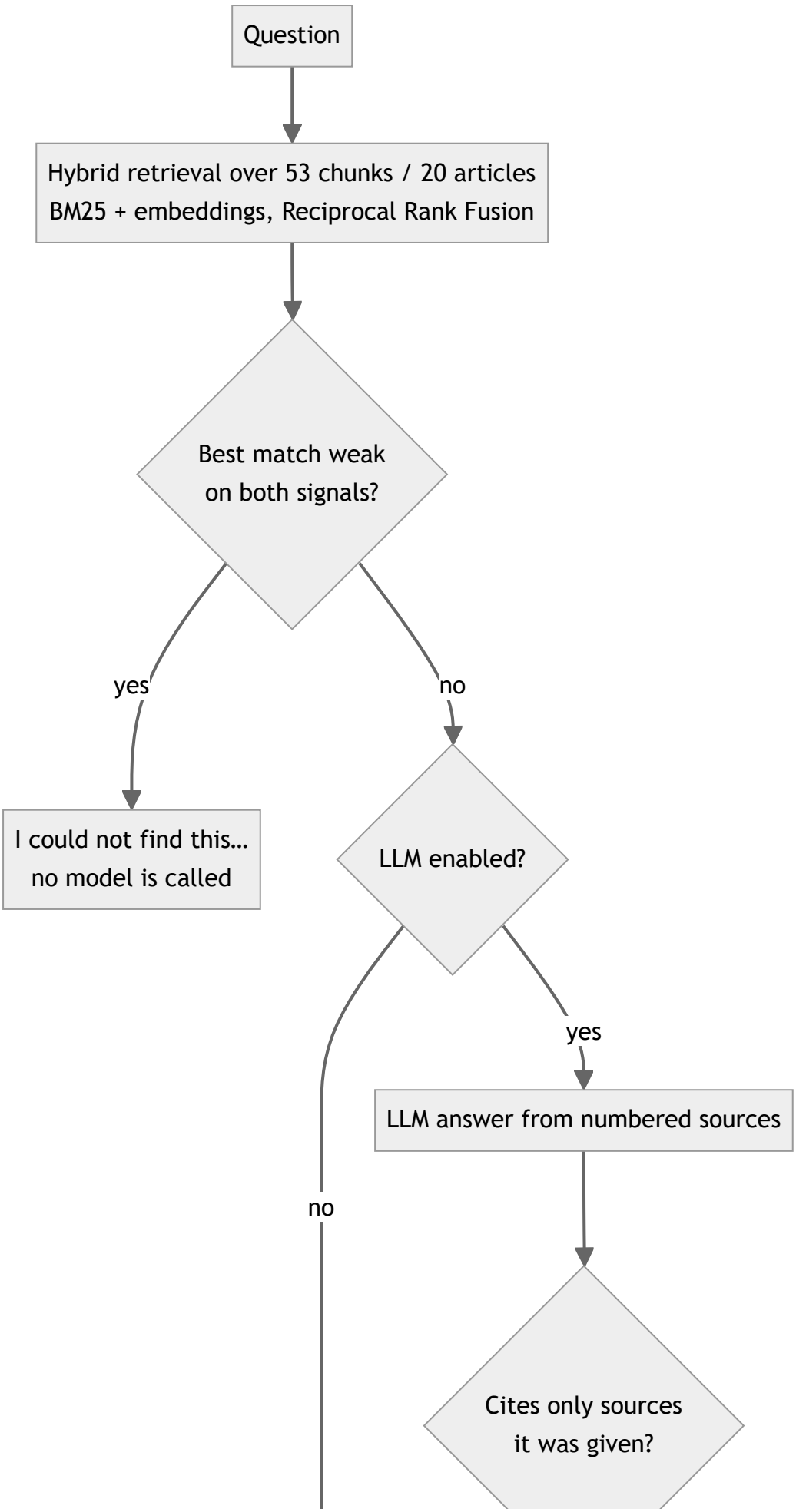
A retrieval-augmented (RAG) support assistant that answers students' questions about a (fictional) university Placement Cell from its help-centre articles. Every answer shows **the articles it came from**, and when the help centre does not cover a question it says **"I could not find this"** instead of guessing. It is measured on a held-out question set, and CI fails the build if retrieval quality drops.

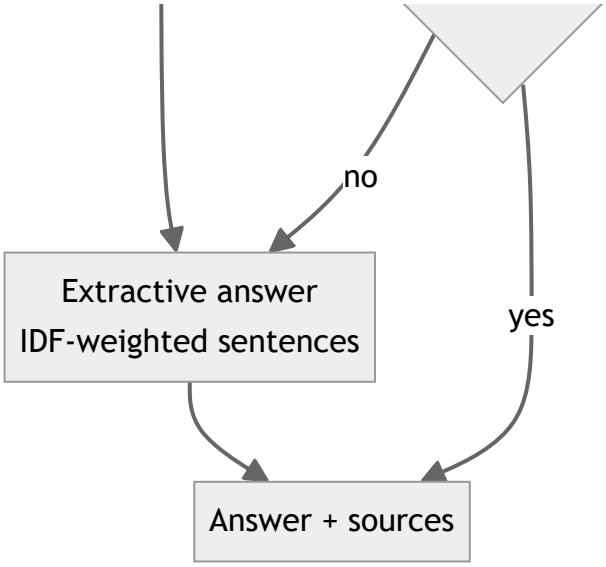
96% right article in the top 3 (held-out test)	**0.92** MRR, hybrid retrieval	**1/27** answerable questions wrongly refused	**15** tests + an evaluation gate in CI



The screenshot shows two answered questions with their sources and one out-of-scope question (hostel curfew) that is refused.

2 How a question is answered


Question
Hybrid retrieval over 53 chunks / 20 articles
BM25 + embeddings, Reciprocal Rank Fusion
Best match weak
on both signals?
yes
no
I could not find this…
no model is called
LLM enabled?
yes
LLM answer from numbered sources
no
Cites only sources
it was given?



1. **Chunking**: one chunk per article section; long sections split into 120-word windows with 30-word overlap. Each chunk keeps its article and section, so it can be cited.
2. **Keyword search (BM25)**, written out by hand: rare words count more (IDF), repeated words saturate (`k1`), long chunks are not favoured (`b`).
3. **Semantic search**: cosine similarity over embeddings. Offline default: LSA (TF-IDF + truncated SVD); pretrained sentence-transformers or an embeddings API are drop-in options.
4. **Reciprocal Rank Fusion** merges the two rankings by position, so their different score scales never have to be compared.
5. **Refuse rule**: if the best chunk is weak on *both* signals (dense < 0.98 and BM25 < 3.5, chosen on the dev split), the assistant refuses and no model is called.
6. **Answer**: extractive by default (the most informative sentences, weighted by IDF). With an LLM enabled, the model gets numbered sources and must cite only those numbers; otherwise the extractive answer is used.

3 Evaluation

46 questions written for this project: 37 answerable (each labelled with the article that answers it and one key fact) and 9 the help centre does not cover. They are deliberately paraphrased ("uncleared subject" instead of "backlog") and split into **dev (14)** for choosing thresholds and **test (32)** for reporting.

Metric (test split, LSA embedder, extractive answers)	Value
Right article ranked first (recall@1), hybrid	89% (24/27)
Right article in the top 3 (recall@3)	96% (26/27)
MRR	0.92
Answer correct: right article **and** the key fact	70% (19/27)
Unanswerable questions correctly refused	3/5
Answerable questions wrongly refused	1/27

What the numbers say

- **Retrieval is strong, sentence choice is weak.** Most wrong answers found the right article but quoted the wrong sentence. The LLM answer step is the natural fix; it is implemented but not yet evaluated, so no number is claimed for it.
- **Refusing is hard with LSA.** LSA similarities are high for almost every chunk, so in practice the keyword score decides refusals. Pretrained embeddings should separate in-scope from out-of-scope questions better.
- **BM25, dense and hybrid score the same here.** 20 short articles full of distinctive words suit keywords; dense retrieval should matter more on larger, vaguer collections.

> The articles and questions were written by the same person, and the test set is small (one question ≈ 3 points), so the scores are optimistic. The design was revised once after the first test run (IDF-weighted sentence choice, light stemming); before that, recall@1 was 85% and answer accuracy 67%. No further tuning used the test split.

4 API and operations

Endpoint	What
`GET /`	Chat page (shareable: `/?q=your+question` asks it on load)
`POST /ask`	`{"question": "..."}` → answer text, sources, whether it refused, mode
`GET /metrics`	Questions answered vs refused, refusal rate, p50/p95 latency
`GET /health`	Chunk count, embedder, the thresholds in use

- **Privacy**: one JSON log line per question with outcome, sources and latency, never the question text, which may contain personal data.
- **Tracing**: `X-Request-ID` on every response.
- **CI**: tests, then the evaluation; the build fails if hybrid recall@3 falls below 0.90; then the Docker image is built.
- **Deploy**: `render.yaml` blueprint for a one-click demo on Render.

5 Code map

File	What
`assistant/chunking.py`	Section chunks with overlap; each keeps its source
`assistant/embeddings.py`	Tokeniser (stopwords, light stemming) and the `lsa` / `sbert` / `api` embedders
`assistant/retriever.py`	BM25, cosine search, Reciprocal Rank Fusion, refuse rule
`assistant/answer.py`	Extractive answers; LLM answers validated against the given sources
`assistant/core.py` , `api.py` , `cli.py`	Wiring, HTTP API and chat page, command line
`eval/run_eval.py`	Threshold search on dev, metrics on test

```
pip install -r requirements.txt
python -m assistant.cli "Can I withdraw after my interview is scheduled?"
uvicorn assistant.api:app --reload     # chat page at http://127.0.0.1:8000/
python -m pytest -q                    # 15 tests
python eval/run_eval.py                # evaluation (offline)
```

6 Next steps

- Evaluate the LLM answer mode and pretrained embeddings on the same test split.
- Real documents (with permission) and questions written by someone other than the author.
- Feedback buttons on answers to collect real failure cases; conversation memory.

7 Interview questions

What do BM25's `k1` and `b` do?

`k1` limits how much repeating a word keeps adding; `b` controls how much long chunks are penalised. IDF makes rare words count more than common ones.

Why fuse by rank instead of adding scores?

BM25 scores are unbounded and cosine scores lie between −1 and 1; adding them lets one dominate. Rank fusion only uses positions.

Why choose thresholds on dev and report on test?

Tuning on the test set makes the reported numbers describe that set, not new questions (overfitting to the test set).

Why check an LLM's citations?

A model can cite sources it was never given or invent facts. Accepting only citations to the provided chunks keeps answers traceable.

Why refuse instead of guessing?

A wrong answer about placement rules can cost a student an opportunity; "I could not find this, contact the Placement Cell" is safer.